TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN — BASES DE DATOS II (UTN FRM)

Informe post-incidente — TP Unidad 6: Mantenimiento, Respaldo y Recuperación

Alumno: Fabrizio Simon | Motor: PostgreSQL 18.6 | Fecha: 10/11/2026 | Proyecto: Food Store

1. DATOS DEL RESPALDO Y DEL ENTORNO DE PRUEBAS

Base Principal: foodstore (96 MB, +470k filas) | **Base Descartable:** foodstore_restore_test | **Archivo:** foodstore_20261110_155301.dump | **Formato:** Custom (-Fc) | **Tamaño:** 3.58 MB (3,752,831 B) | **Backup:** 10/11/2026 15:53:03 | **Simulacro:** 10/11/2026 15:54:45

2. TIEMPOS Y MÉTRICAS: RTO Y RPO (OBJETIVO VS. LO QUE ME DIO EN EL LAB)

Métrica de Continuidad	Qué mide / Criterio	Objetivo Previsto	Lo que me dio en el Lab	Conclusión
RTO (Tiempo recuperación)	De punta a punta: createdb + pg_restore + validaciones SQL.	≤ 5 minutos (300 s)	**4.84 segundos**	**Cumplido sobradísimo**
RPO (Pérdida de datos)	Antigüedad del volcado al momento de iniciar la recuperación.	≤ 24 horas (política)	**1.70 min (102 s)**	**Cumplido (0 pérdida)**

3. VALIDACIONES DE DATOS Y REGLAS LÓGICAS SOBRE FOODSTORE_RESTORE_TEST

Tabla / Regla de Integridad	Filas en Origen	Filas Restauradas	Registros Inválidos	Estado
categoria (catálogo de rubros)	10	10	0	■ **Idéntico**
cliente (maestro de clientes)	20,001	20,001	0	■ **Idéntico**
producto (precio >= 0 AND stock >= 0)	50,002	50,002	**0 inv. (precio/stock < 0)**	■ **100% íntegro**
pedido (cabeceras transaccionales)	200,001	200,001	0	■ **Idéntico**
detalle_pedido (cant > 0 AND precio >= 0)	200,000	200,000	**0 inv. (cant/precio <= 0)**	■ **100% íntegro**

4. PROBLEMA REAL QUE ME ENCONTRÉ DURANTE EL PROCEDIMIENTO

Al principio, cuando corrí pg_restore pelado sin flags adicionales, el comando me tiraba advertencias de permisos y ownership porque intentaba recrear los mismos dueños y grants específicos de los roles del servidor de origen, que en una base de prueba descartable no hacen falta o pueden no existir. **Cómo lo solucioné:** Le agregué obligatoriamente las banderas --no-owner --no-privileges al comando dentro de restore_test.ps1. Con eso la restauración entró limpia y sin errores de permisos. También me aseguré de mapear con una vista de compatibilidad detalle_pedido sobre pedido_producto y la columna precio para que las consultas de validación de la cátedra corrieran sin tocar nada raro.

5. MEJORAS CONCRETAS PARA LA PRÓXIMA EJECUCIÓN

1. Restaurar con jobs en paralelo (-j): Si la base en vez de 96 MB llega a pesar 20 o 50 GB, agregaría pg_restore -j 4 para restaurar tablas e índices en paralelo usando varios núcleos del CPU y mantener el RTO bajo control.
2. Archivado continuo de WAL: Para producción real, no alcanza solo con el backup de la noche; activaría archive_mode = on con archivado de WAL para tener Point-in-Time Recovery (PITR) y bajar el RPO real a menos de 5 minutos ante cualquier caída.
3. Alerta automática si falla: En backup_foodstore.ps1, enganchar una notificación automática (un webhook a Discord/Slack o mail) si $LASTEXITCODE != 0, así no dependés de entrar a revisar la carpeta logs/ a mano.

6. ASISTENCIA DE OPENCODE Y VALIDACIÓN HUMANA

Usé OpenCode para generar la primera versión de los scripts de PowerShell y las consultas de auditoría. Ninguna sugerencia se aceptó a ciegas:
• *Contraseñas:* OpenCode me proponía dejar un parámetro con la password en texto plano. Lo saqué de raíz y forcé que use $env:PGPASSWORD.
• *Mantenimiento seguro:* Descarté de plano la idea de meter VACUUM FULL semanal porque te mete un bloqueo exclusivo (AccessExclusiveLock) y te frena la base; dejé VACUUM (ANALYZE, VERBOSE) ordinario que limpia tuplas muertas en caliente sin afectar a los usuarios.
• *Medición real:* En vez de estimar tiempos aproximados, armé el script restore_test.ps1 con un cronómetro real de sistema para medir el RTO exacto con evidencia en log.